



JavaScript

Ejercicios

Ejercicio 1

Escribir una función **producto** que reciba dos parámetros (llamados **x** e **y**) y devuelva su producto, teniendo en cuenta que tanto la **x** como la **y** pueden ser números o vectores (representados como arrays). La función se comportará del siguiente modo:

- Si **x** e **y** son números, se calculará su producto.
- Si **x** es un número e **y** es un vector (o viceversa), se calculará el vector que resulta de multiplicar todas las componentes de **y** por **x**.
- Si **x** e **y** son vectores de la misma longitud, se calculará el producto escalar de ambos. Se lanzará una excepción si los dos vectores no son de la misma longitud.
- En cualquier otro caso, se lanzará una excepción.

Ejercicio 2

Escribir una función **sequence** que reciba un array de funciones [**f_1**, ..., **f_n**] y un elemento inicial **x**. La función debe aplicar **f_1** a **x**, y pasar el resultado a **f_2** que a su vez le pasará el resultado a **f_3** y así sucesivamente. Se piden tres versiones diferentes de la función **sequence**:

- (a) Implementar la función **sequence1** suponiendo que ninguna de las funciones del array recibido devuelve el valor undefined.
- (b) Implementar la función **sequence2** para que, si una función del array devuelve el valor undefined, la función **sequence2** devuelva directamente undefined sin seguir ejecutando las funciones restantes.
- (c) Implementar la función **sequence3** para que reciba un tercer parámetro opcional (**right**), cuyo valor por defecto será **false**. Si el parámetro **right** tiene el valor **true**, el recorrido del elemento por las funciones será en orden inverso: desde la última función del array hasta la primera. Independientemente del recorrido, la función **sequence3** se comportará como la función **sequence2**.

Ejercicio 3

- (a) Escribir una función **pluck(objects, fieldName)** que devuelva la propiedad de nombre **fieldName** de cada uno de los objetos contenidos en el array **objects** de entrada. Se devolverá un array con los valores correspondientes en el mismo orden de aparición que en el array de entrada. Se deberá contemplar el caso en el que alguno de los objetos no contenga el atributo dado. Si



ninguno de los objetos contienen la propiedad dada, la función devolverá un array vacío. Por ejemplo:

```
let personas = [  
  {nombre: "Ricardo", edad: 63},  
  {nombre: "Paco", edad: 55},  
  {nombre: "Enrique", edad: 32},  
  {nombre: "Adrián", edad: 34},  
  {apellido: "García", edad: 28}  
];  
pluck(personas, "nombre") // Devuelve: ["Ricardo", "Paco", "Enrique", "Adrián"]  
pluck(personas, "edad") // Devuelve: [63, 55, 32, 34, 28]  
pluck(personas, "apellido") // Devuelve: ["García"]  
pluck(personas, "email") // Devuelve: []
```

- (b) Implementar una función **partition(array, p)** que devuelva un array con dos arrays. El primero contendrá los elementos **x** de **array** tales que **p(x)** devuelve true. Los restantes elementos se añadirán al segundo array. Por ejemplo:

```
partition(personas, pers => pers.edad >= 60)  
// Devuelve:  
// [  
// [ {nombre: "Ricardo", edad: 63} ],  
// [ {nombre: "Paco", edad: 55}, {nombre: "Enrique", edad: 32},  
// {nombre: "Adrián", edad: 34}, {apellidos: "García", edad: 28 }  
// ]
```

- (c) Implementar una función **groupBy(array, f)** que reciba un **array**, una función clasificadora **f**, y reparta los elementos del **array** de entrada en distintos arrays, de modo que dos elementos pertenecerán al mismo array si la función clasificadora devuelve el mismo valor para ellos. Al final se obtendrá un objeto cuyas propiedades son los distintos valores que ha devuelto la función clasificadora, cada uno de ellos asociado a su array correspondiente. Ejemplo:

```
groupBy(["Mario", "Elvira", "María", "Estela", "Fernando"],  
str => str[0]) // Agrupamos por el primer carácter  
// Devuelve el objeto:
```



```
// { "M" : ["Mario", "María"], "E" : ["Elvira", "Estela"], "F" : ["Fernando"] }
```

- (d) Escribir una función **where(array, modelo)** que reciba un **array** de objetos y un objeto **modelo**. La función ha de devolver en un array aquellos objetos del **array** que contengan todas las propiedades contenidas en **modelo** con los mismos valores. Ejemplo:

```
where(personas, { edad: 55 })  
// devuelve [ { nombre: 'Paco', edad: 55 } ]
```

```
where(personas, { nombre: "Adrián" })  
// devuelve [ { nombre: 'Adrián', edad: 34 } ]
```

```
where(personas, { nombre: "Adrián", edad: 21 })  
// devuelve []
```

Ejercicio 4

Reescribir las funciones del ejercicio anterior pero utilizando exclusivamente funciones de orden superior.

Ejercicio 5

Escribir una función **concatenar** que permita concatenar un número no predefinido de cadenas de caracteres utilizando un carácter separador. El primer parámetro de la función es el carácter separador. Este parámetro va seguido de las cadenas que se concatenarán. Se espera el siguiente comportamiento de la función:

```
concatenar()           // Devuelve la cadena vacía  
concatenar("-")        // Devuelve la cadena vacía  
concatenar("-", "uno") // Devuelve "uno-"  
concatenar("-", "uno", "dos", "tres") // Devuelve "uno-dos-tres-"
```

Ejercicio 6



Escribe una función **mapFilter(array, f)** que se comporte como **map**, pero descartando los elementos obj de la entrada tales que f(obj) devuelve undefined. Por ejemplo:

```
mapFilter(["23", "44", "das", "555", "21"],  
(str) => {  
  let num = Number(str);  
  if (!isNaN(num)) return num;  
})  
// Devuelve: [23, 44, 555, 21]
```

Ejercicio 7

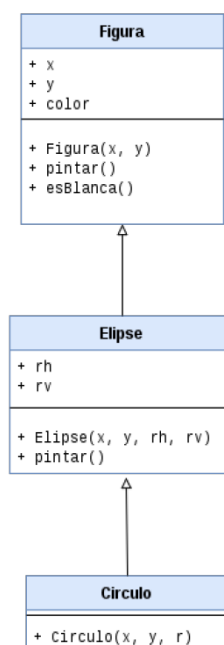
Utilizando expresiones regulares, implementar la función **interpretarColor(str)** que, dada una cadena que representa un color en formato hexadecimal #RRGGBB devuelva un objeto con tres atributos (**red**, **green** y **blue**) con el valor (en base 10) de la componente correspondiente.

Si la cadena de entrada no es un color en formato hexadecimal válido, la función devuelve null.

Indicación: se puede utilizar **parseInt** para realizar conversiones entre distintas bases.

Ejercicio 8

En la Figura se muestra el diagrama correspondiente a tres clases: **Figura**, **Elipse** y **Circulo**.





Los objetos de la clase **Figura** representan figuras geométricas. Cada uno de ellos tiene asignada una posición (coordenadas **x** e **y**) y un color. Todas las propiedades son de lectura y de escritura, pero solamente se permitirá escribir en el atributo **color** si el valor escrito es una cadena que contenga un color HTML en notación hexadecimal (por ejemplo, **"#23B5DD"**). El valor por defecto del color es **"#000000"** (los caracteres hexadecimales pueden aparecer en mayúsculas o en minúsculas).

El método **esBlanca** devuelve **true** cuando se llama sobre un objeto cuyo **color** es **"#FFFFFF"**. El método **pintar** imprime el siguiente texto por la consola,

Nos movemos a la posición ([x], [y])
Cogemos la pintura de color [color]

sustituyendo **[x]**, **[y]** y **[color]** por sus respectivos valores.

Los objetos **Elipse** heredan de **Figura** y contienen dos atributos adicionales: **rh** (radio horizontal) y **rv** (radio vertical). El método **pintar** debe imprimir lo siguiente:

Nos movemos a la posición ([x], [y])
Cogemos la pintura de color [color]
Pintamos elipse de radios [rh] y [rv]

Por último, los objetos **Circulo** representan elipses en las que $rh = rv$.

Se pide representar esta jerarquía de clases en Javascript con la notación ES6.